

UNIVERSITY OF MINES AND TECHNOLOGY, TARKWA
FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES
DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEMS



MONITORING VPN ACCESS LOGS FOR ANOMALOUS REMOTE CONNECTIONS: A RULE-BASED DETECTION PIPELINE FOR OPENVPN

(BLUE TEAM)

NETWORK MONITORING, SECURITY AND AUDITING

Name: Appiah Felix

Index Number: FCM.41.018.066.23

Course Code: CY376

Date: Monday, 3 August 2026

GitHub Repository: https://github.com/AppiahFelix/access_logs

Abstract

A Virtual Private Network gives remote users encrypted access into a private network, but that same convenience is what makes it a favoured entry point once a set of credentials has been guessed, phished, or reused from another breach. MITRE ATT&CK tracks this as a distinct technique, External Remote Services, precisely because a valid VPN login looks identical to an attacker and to a legitimate employee unless something is comparing the event against a baseline. Most small organizations, and most class-lab OpenVPN deployments, keep a connection log and nothing else; nobody reads it unless something has already gone wrong.

This project builds a small, rule-based monitoring pipeline that sits on top of an OpenVPN server and turns its raw connection log into an analyst-facing anomaly dashboard. An OpenVPN gateway was deployed inside Docker on a macOS host, with connect and disconnect hooks writing a structured event for every session. A normalization script enriches each event with geolocation from a local MaxMind GeoLite2 database, and a detection engine applies five explainable rules: brute-force login attempts, impossible travel between two logins, access outside normal business hours, a login from a country the user has never connected from before, and a session that runs far longer than that user typically keeps one open. A static HTML dashboard renders the findings without needing a server of its own.

The pipeline was tested against a real, if modest, dataset: thirty-two logged events collected across several client identities over roughly three weeks, including genuine reconnects, a laptop switch, and late-evening development sessions. Running the detectors against that data flagged eight anomalies: one unusual-country finding and seven off-hours findings; the remaining three detectors stayed silent, and the report explains why that silence is itself a correct result rather than a gap. The main limitation encountered, Docker Desktop's network address translation collapsing every client's visible source address to the same local gateway IP, is discussed openly, along with what changes in a deployment where the server sits on a routable address instead of behind a lab NAT.

TABLE OF CONTENTS

ABSTRACT	1
CHAPTER 1: INTRODUCTION	3
CHAPTER 2: LITERATURE AND TOOLING REVIEW.....	4
2.1 Attacker techniques the project is defending against	4
2.2 Standards for logging and remote-access hygiene	4
2.3 Tooling choices	4
2.4 Why rule-based detection rather than machine learning	4
CHAPTER 3: METHODOLOGY	5
3.1 Lab environment	5
3.2 Steps taken	5
3.3 Architecture	5
CHAPTER 4: IMPLEMENTATION.....	7
4.1 Server and client provisioning	7
4.2 Event logging hooks	7
4.3 Normalization layer	8
4.4 Detection engine	8
4.5 Dashboard	9
CHAPTER 5: RESULTS AND FINDINGS	10
CHAPTER 6: ANALYSIS AND RECOMMENDATIONS	15
6.1 Reading the results together	15
6.2 The central limitation	15
6.3 Recommendations	15
6.4 Why the rule-based approach held up	16
CHAPTER 7: CONCLUSION	17
CHAPTER 8: REFERENCES	18
Appendix A: Selected Script Listings	19
A.1 client-connect.sh	19
A.2 client-disconnect.sh	19
A.3 watch_failed_auth.sh	19
Appendix B: Raw Event Log	20

1. Introduction

Remote access is one of the more predictable places to look for a way into an otherwise well-defended network. A firewall can block almost everything except the one service designed to let people in from outside, and that is exactly where a VPN gateway sits. MITRE's ATT&CK framework lists External Remote Services (T1133) as a standing initial-access technique for this reason, and Valid Accounts (T1078) as the natural follow-on once a credential has been obtained: an attacker who reaches a VPN login page with a stolen or guessed password does not need to exploit anything, they only need the network to treat them the way it treats everyone else who authenticates correctly.

The practical problem is that most small deployments do not have anyone looking at the log. An OpenVPN server writes to syslog by default, and syslog is not something a small team or a solo administrator checks daily. A brute-force attempt spread out over an evening, a login from a country nobody on the team has ever visited, or an account that connects at three in the morning all look, at the level of a single log line, exactly like ordinary traffic. The only way to catch any of them is to compare each new event against a baseline of what normal looks like for that particular user, and that comparison has to happen automatically or it will not happen at all.

This project sets out to build that comparison layer for OpenVPN specifically, as a self-contained, inspectable pipeline rather than a commercial SIEM integration. The scope is deliberately small: a single Docker-hosted OpenVPN gateway, a handful of client identities, and five rule-based detectors chosen because each one maps to a recognisable attack pattern and each one can be explained in a single sentence to whoever has to trust its output. The project also had to work with what the lab environment actually produced rather than an idealised dataset, which meant confronting a genuine limitation of running the server inside Docker Desktop on a laptop, described in detail in the Methodology and Analysis sections.

The remainder of this report is organised as follows. Section 2 reviews the frameworks and tools that shaped the approach. Section 3 describes the lab setup and the steps taken to build it. Section 4 documents what was implemented, with configuration and code excerpts. Section 5 presents the results of running the pipeline against real logged connections. Section 6 analyses those results and sets out recommendations. Section 7 concludes.

2. Literature and Tooling Review

2.1 Attacker techniques the project is defending against

MITRE ATT&CK's External Remote Services (T1133) describes adversaries using a VPN, RDP gateway, or similar service as their point of entry, and Valid Accounts (T1078) describes what they do once a credential works: log in normally and blend in with legitimate traffic [1], [2]. Brute Force (T1110) covers the credential-guessing stage that often precedes both [3]. These three techniques between them cover the majority of what a VPN-focused monitoring layer is realistically trying to catch, and they map directly onto three of the five detectors implemented here: repeated failed logins, an unfamiliar login pattern, and continued access after that pattern shows up.

2.2 Standards for logging and remote-access hygiene

The CIS Critical Security Controls place both access-control management and audit-log management among their core safeguards, on the reasoning that a log nobody reviews provides no actual security benefit regardless of how complete it is [4]. NIST's guide to enterprise telework and remote access similarly recommends that organisations monitor remote-access connections for anomalous patterns rather than treating a successful VPN handshake as the end of the security conversation [5]. Both sources point at the same gap this project tries to close: the difference between having a log and having something that reads the log.

2.3 Tooling choices

OpenVPN was chosen as the VPN implementation because it is open source, uses certificate-based mutual authentication by default, and exposes client-connect and client-disconnect hook scripts that fire automatically on every session change [6]. That hook mechanism is what makes structured logging possible without patching OpenVPN itself. The server was run through the kylemanna/openvpn Docker image rather than installed directly on a cloud VM, which kept the lab reproducible and let the Easy-RSA certificate authority live inside the container instead of on the host.

For geolocation, MaxMind's GeoLite2 City database was used in preference to a live API call against a third-party geolocation service [7]. A local database means IP lookups happen offline, are not rate-limited, and do not send connecting users' addresses to an external party for every login; the normalisation script falls back to the free ip-api.com service only when no local database is present, which keeps the pipeline runnable without a MaxMind account but is not the recommended path for anything beyond a first test.

2.4 Why rule-based detection rather than machine learning

A rule-based detector was chosen deliberately over a statistical or machine-learning anomaly model. Every one of the five rules implemented can be stated in one sentence, tuned by changing a single threshold, and audited by a person who is not a data scientist, which matters when the team relying on the output is the same team that has to maintain it. For a first monitoring pass over a VPN gateway, explainability was judged more valuable than sensitivity.

3. Methodology

3.1 Lab environment

The lab was built entirely on a single macOS host (a MacBook Air) running Docker Desktop. The OpenVPN server ran inside the kylemanna/openvpn container image, which bundles OpenVPN itself alongside Easy-RSA for certificate management. Client-side connections were made using Tunnelblick, the standard OpenVPN client for macOS. Three client identities were provisioned for testing, user1 through user3, in addition to a pre-existing production-style identity (joe, later re-issued as joe-laptop and joe-laptop2 after a device change) that already had roughly three weeks of real connection history by the time the detection engine was run against it.

3.2 Steps taken

1. Deployed the OpenVPN server container and generated the certificate authority and per-client certificates through the container's built-in Easy-RSA tooling.
2. Installed client-connect.sh and client-disconnect.sh as hook scripts referenced from server.conf, with script-security set to 2, and restarted the OpenVPN service so every future session change would be logged automatically.
3. Deployed watch_failed_auth.sh as a background tailer against the server's own log, to capture failed-authentication attempts, which do not pass through the connect hook because that hook only fires on success.
4. Connected each client identity across multiple sessions, at different times of day and after at least one real device change (joe to joe-laptop to joe-laptop2), so the resulting log would reflect genuine usage rather than a single artificial batch of test data.
5. Downloaded GeoLite2-City.mmdb from MaxMind and placed it alongside the normalisation script so IP-to-location lookups would run locally rather than depend on a rate-limited public API.
6. Ran normalize_openvpn_logs.py against the accumulated events.log to produce vpn_logs.csv in the schema the detection engine expects.
7. Ran analyze_logs.py against vpn_logs.csv to produce anomalies.json and dashboard_data.js.
8. Opened dashboard.html in a browser to review the flagged events and confirm the dashboard rendered the same findings written to anomalies.json.

3.3 Architecture

Figure 1 shows the resulting pipeline end to end: clients authenticate against the OpenVPN container over a certificate-based tunnel; the connect and disconnect hooks append a structured line to events.log; the normalisation script enriches each line with location data and writes it to a common CSV schema; the detection engine reads that CSV, applies its five rules, and writes both a JSON findings file and a JavaScript data file; and the dashboard loads that data file directly in the browser, with no server component of its own.

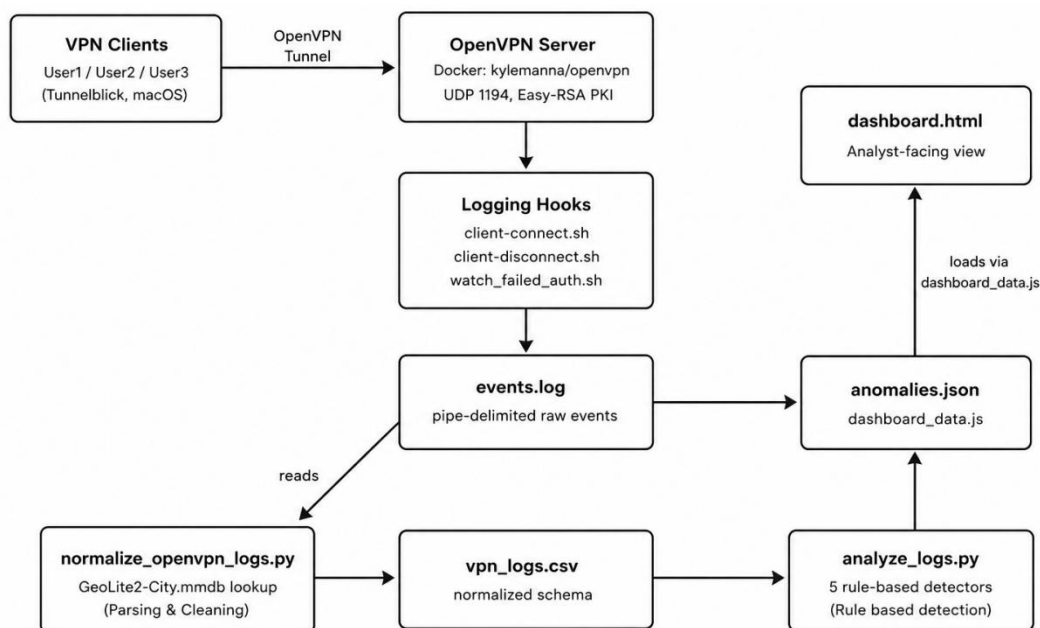


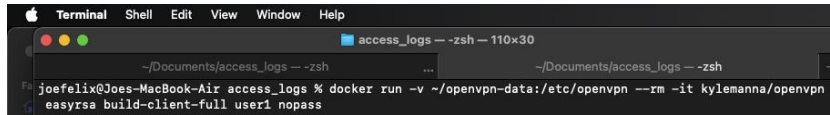
Figure 1. End-to-end pipeline: client authentication, event logging, normalisation, rule-based detection, and dashboard rendering.

One methodological limitation surfaced during this stage and shaped several of the results discussed in Section 5: Docker Desktop on macOS routes every container-bound connection through the host's network address translation. In practice this means every client, regardless of which physical network it actually connected from, arrives at the OpenVPN server showing the same local gateway address. This is discussed further in Section 6, since it directly affects two of the five detectors.

4. Implementation

4.1 Server and client provisioning

The OpenVPN server and its three test clients were provisioned entirely through Docker, keeping the certificate authority, server configuration, and per-client keys inside a single mounted volume. Client certificates were generated with Easy-RSA's build-client-full command, one call per identity:

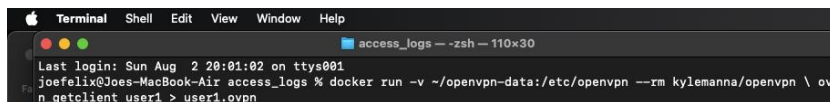


```

access_logs -- zsh -- 110x30
~/Documents/access_logs -- zsh
joefelix@Joes-MacBook-Air access_logs % docker run -v ~/openvpn-data:/etc/openvpn --rm -it kylemanna/openvpn \
easyrsa build-client-full user1 nopass

```

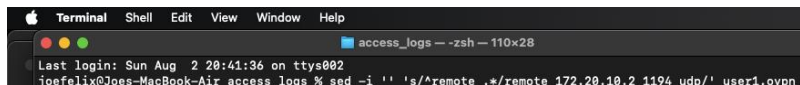
Each client configuration was then exported and had its remote address corrected to point at the gateway's current network address, since a laptop-hosted lab server does not sit at a fixed address the way a cloud VM would:



```

access_logs -- zsh -- 110x30
Last login: Sun Aug 2 20:01:02 on ttys001
joefelix@Joes-MacBook-Air access_logs % docker run -v ~/openvpn-data:/etc/openvpn --rm kylemanna/openvpn \
n_getclient user1 > user1.ovpn

```



```

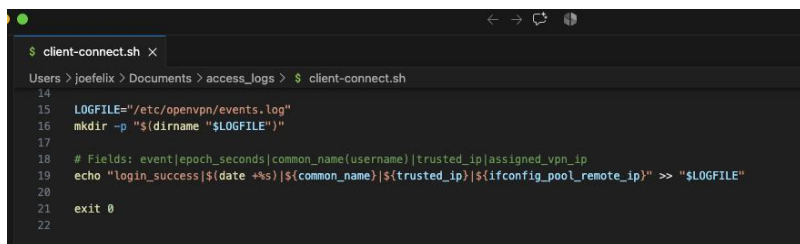
access_logs -- zsh -- 110x28
Last login: Sun Aug 2 20:41:36 on ttys002
joefelix@Joes-MacBook-Air access_logs % sed -i '' 's/^remote .*/remote 172.20.10.2 1194 udp/' user1.ovpn

```

Because the gateway's address changes whenever the host's network changes, a small helper script, `update_ip.sh`, was written to detect the host's current interface address and rewrite every `.ovpn` file in the working directory in one pass, rather than repeating the `sed` command by hand for each client before every test session.

4.2 Event logging hooks

OpenVPN calls `client-connect.sh` automatically on every successful connection and `client-disconnect.sh` on every disconnect, passing connection details as environment variables. Both scripts do nothing more than append one structured, pipe-delimited line to a shared log file, which keeps the logging layer simple and avoids parsing OpenVPN's own noisier syslog output:



```

$ client-connect.sh X
Users > joefelix > Documents > access_logs > $ client-connect.sh
14
15 LOGFILE="/etc/openvpn/events.log"
16 mkdir -p "$(dirname "$LOGFILE")"
17
18 # Fields: event|epoch_seconds|common_name(username)|trusted_ip|assigned_vpn_ip
19 echo "login_success|$(date +%s)|${common_name}|${trusted_ip}|${ifconfig_pool_remote_ip}" >> "$LOGFILE"
20
21 exit 0
22

```

The disconnect hook writes the same fields plus a session duration in seconds, which the normalisation step later converts to minutes for the long-session detector. Failed authentication attempts do not trigger either hook, since `client-connect` only fires once a session has already been accepted, so a separate script, `watch_failed_auth.sh`, tails the server's own log in the background and pattern-matches lines that look like an authentication failure, writing a `login_failed` event in the

same schema when it finds one. Because the exact wording of a failure line depends on which authentication method the server is configured for, this script is explicitly documented in its own comments as a best-effort heuristic that should be checked against a handful of real failed attempts before being trusted in a new environment.

4.3 Normalisation layer

normalize_openvpn_logs.py is the single integration point between whatever a given OpenVPN deployment logs and the detection engine, which never has to change regardless of the log source. It reads the pipe-delimited events.log, geolocates every source IP, and writes a ten-column CSV: timestamp, username, event, source_ip, country, city, lat, lon, vpn_gateway, and session_duration_min. Private and locally assigned addresses, which a home lab or a NAT'd container will produce constantly, are detected up front and labelled Local/VPN-assigned rather than sent through a geolocation lookup that would return nothing useful for them.

4.4 Detection engine

analyze_logs.py implements five independent, threshold-driven detectors, summarised in Table 1. Each detector is a short, self-contained function that reads the normalised CSV once and appends any findings to a shared list, which keeps every rule auditable on its own without needing to trace logic across the rest of the file.

Detector	Rule	Severity
Brute force	Five or more failed logins against the same username and source IP within a 15-minute window.	High
Impossible travel	Two successful logins for the same user, more than 300 km apart, imply a travel speed above 900 km/h.	High
Off-hours access	Successful login outside the 07:00-19:00 window.	Medium
Unusual country	First-ever successful login from a country not previously seen for that user.	High
Long session	Session length exceeds 8 hours and is at least three times that user's median session length.	Low

Table 1. Detector thresholds implemented in analyze_logs.py.

The impossible-travel detector is the most involved of the five, since it needs a distance calculation between two geographic points rather than a simple field comparison:

```

47 def haversine_km(lat1, lon1, lat2, lon2):
48     R = 6371.0
49     phi1, phi2 = math.radians(lat1), math.radians(lat2)
50     dphi = math.radians(lat2 - lat1)
51     dlambda = math.radians(lon2 - lon1)
52     a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlambda / 2) ** 2
53     return 2 * R * math.asin(math.sqrt(a))
54

```

Every finding, regardless of which detector produced it, is written in the same shape: a type, a severity, the username and source IP involved, a timestamp, and a short human-readable detail string. That consistency is what lets a single dashboard render all five detectors' output without needing detector-specific display logic.

4.5 Dashboard

dashboard.html is a single static file with no server component: it loads dashboard_data.js, a small JavaScript file that analyze_logs.py regenerates on every run, and renders four elements from it, a live-updating anomaly stream ordered most-recent-first, summary counters split by severity, a login-volume chart comparing successful and failed attempts per day, and a filterable table of every flagged connection. Because the dashboard only reads a locally generated file, it can be reviewed simply by re-opening dashboard.html after each run of the analysis script, with no deployment step of its own.

5. Results and Findings

The pipeline was exercised against real connection history rather than a purely synthetic dataset. By the time the detection engine was run, events.log held thirty-two entries spanning 13 July through 1 August, covering the original joe identity, its two later re-issues after a device change (joe-laptop, joe-laptop2), and the three test identities (user1 through user3) created for this project. Figures 2 through 8 walk through the pipeline running end to end against that data.

```

Last login: Sat Aug  1 22:19:24 on ttys001
joe@joe-MacBook-Air:~$ ipconfig getifaddr en0
172.20.10.2
joe@joe-MacBook-Air:~$ rm joe.ovpn joe-laptop.ovpn joe-laptop2.ovpn broken.ovpn
joe@joe-MacBook-Air:~$ docker run -v ~/openvpn-data:/etc/openvpn --rm -it kylemanna/openvpn easyrsa build-client-full user1 nopass

Using SSL: openssl OpenSSL 1.1.1g  21 Apr 2020
Generating a RSA private key
.....+++++
.....+++++
writing new private key to '/etc/openvpn/pki/easy-rsa-1.HmcfpC/tmp.FHCojK'
-----
Using configuration from /etc/openvpn/pki/easy-rsa-1.HmcfpC/tmp.EF1FgN
Enter pass phrase for /etc/openvpn/pki/private/ca.key:
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
commonName            :ASN.1 12:'user1'
Certificate is to be certified until Nov  3 22:37:03 2028 GMT (825 days)

Write out database with 1 new entries
Data Base Updated

joe@joe-MacBook-Air:~$ docker run -v ~/openvpn-data:/etc/openvpn --rm -it kylemanna/openvpn easyrsa build-client-full user2 nopass

Using SSL: openssl OpenSSL 1.1.1g  21 Apr 2020
Generating a RSA private key
.....+++++
.....+++++
writing new private key to '/etc/openvpn/pki/easy-rsa-1.Eajlno/tmp.JKoFCo'
-----
Using configuration from /etc/openvpn/pki/easy-rsa-1.Eajlno/tmp.HngBpn
Enter pass phrase for /etc/openvpn/pki/private/ca.key:
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
commonName            :ASN.1 12:'user2'
Certificate is to be certified until Nov  3 22:37:31 2028 GMT (825 days)

Write out database with 1 new entries
Data Base Updated

joe@joe-MacBook-Air:~$ docker run -v ~/openvpn-data:/etc/openvpn --rm -it kylemanna/openvpn easyrsa build-client-full user3 nopass

Using SSL: openssl OpenSSL 1.1.1g  21 Apr 2020
Generating a RSA private key
.....+++++
.....+++++

```

Figure 2. Client certificate generation for user1, user2, and user3 through the container's built-in Easy-RSA tooling.

Figure 2 shows the three test identities being issued in sequence, each producing a distinct commonName in the certificate authority's database. Generating them individually rather than as a batch made it straightforward to confirm the CA's passphrase prompt behaved consistently and that each certificate carried a distinguishable identity through to the connection logs.

```

writing new private key to '/etc/openvpn/pki/easy-rsa-1.EajLno/tmp.JKoFCo'
-----
Using configuration from /etc/openvpn/pki/easy-rsa-1.EajLno/tmp.HngBpn
Enter pass phrase for /etc/openvpn/pki/private/ca.key:
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
commonName      :ASN.1 12:'user2'
Certificate is to be certified until Nov  3 22:37:31 2028 GMT (825 days)

Write out database with 1 new entries
Data Base Updated

joefelix@Joes-MacBook-Air access_logs % docker run -v ~/openvpn-data:/etc/openvpn --rm -it kylemanna/openvpn easyrsa build-client-full user3 nopass
Using SSL: openssl OpenSSL 1.1.1g  21 Apr 2020
Generating a RSA private key
.....+++++
.....+++++
writing new private key to '/etc/openvpn/pki/easy-rsa-1.fIkGmd/tmp.pnJdPE'
-----
Using configuration from /etc/openvpn/pki/easy-rsa-1.fIkGmd/tmp.jcogeG
Enter pass phrase for /etc/openvpn/pki/private/ca.key:
Check that the request matches the signature
Signature ok
The Subject's Distinguished Name is as follows
commonName      :ASN.1 12:'user3'
Certificate is to be certified until Nov  3 22:37:48 2028 GMT (825 days)

Write out database with 1 new entries
Data Base Updated

joefelix@Joes-MacBook-Air access_logs % docker run -v ~/openvpn-data:/etc/openvpn --rm kylemanna/openvpn ovpn_getclient user1 > user1.ovpn
joefelix@Joes-MacBook-Air access_logs % docker run -v ~/openvpn-data:/etc/openvpn --rm kylemanna/openvpn ovpn_getclient user2 > user2.ovpn
joefelix@Joes-MacBook-Air access_logs % docker run -v ~/openvpn-data:/etc/openvpn --rm kylemanna/openvpn ovpn_getclient user3 > user3.ovpn
joefelix@Joes-MacBook-Air access_logs % sed -i 's/^remote .*/remote 172.20.10.2 1194 udp/' user1.ovpn
joefelix@Joes-MacBook-Air access_logs % sed -i 's/^remote .*/remote 172.20.10.2 1194 udp/' user2.ovpn
joefelix@Joes-MacBook-Air access_logs % sed -i 's/^remote .*/remote 172.20.10.2 1194 udp/' user3.ovpn
joefelix@Joes-MacBook-Air access_logs % grep -n '^remote ' user1.ovpn user2.ovpn user3.ovpn
user1.ovpn:7:remote 172.20.10.2 1194 udp
user2.ovpn:7:remote 172.20.10.2 1194 udp
user3.ovpn:7:remote 172.20.10.2 1194 udp
joefelix@Joes-MacBook-Air access_logs %

```

Figure 3. Certificate export for all three identities and correction of the server address in each resulting configuration file.

Figure 3 continues from the certificate generation shown in Figure 2, exporting each identity to its own .ovpn file and then rewriting the remote line in all three so that they point at the gateway's current network address. The final grep command confirms all three files agree on the same address before any of them are used to connect.

```

~/Documents/access_logs -- zsh
Last login: Sat Aug  1 22:53:34 on console
joefelix@Joes-MacBook-Air ~ % cd ~/Documents/access_logs

joefelix@Joes-MacBook-Air access_logs % ~/Documents/access_logs/update_ip.sh
Current Mac IP: 192.168.0.114
Updated user1.ovpn
Updated user2.ovpn
Updated user3.ovpn
joefelix@Joes-MacBook-Air access_logs % open -a Docker

joefelix@Joes-MacBook-Air access_logs % docker start openvpn-server

openvpn-server
joefelix@Joes-MacBook-Air access_logs %

```

Figure 4. Execution of update_ip.sh to synchronise client configuration files with the host's current network address, followed by starting the OpenVPN server container.

Figure 4 shows the helper script described in Section 4.1 detecting the host's current interface address and updating every .ovpn file in the working directory automatically, followed by the openvpn-server container being started for that session.

```

~/Documents/access_logs -- -zsh
joeefelix@Joes-MacBook-Air access_logs % cat ~/openvpn-data/events.log

logout|1783931025|joe|192.168.65.1|192.168.255.6|239
login_success|1783931026|joe|184.27.140.133|192.168.255.6
logout|1783931763|joe|184.27.140.133|192.168.255.6|737
login_success|1783932407|joe|192.168.65.1|192.168.255.6
logout|1783932538|joe|192.168.65.1|192.168.255.6|131
login_success|1783932538|joe|192.168.65.1|192.168.255.6
logout|1783932598|joe|192.168.65.1|192.168.255.6|60
login_success|1783932598|joe|192.168.65.1|192.168.255.6
login_success|1783934089|joe|192.168.65.1|192.168.255.6
logout|1783934511|joe|192.168.65.1|192.168.255.6|422
login_success|1783935140|joe-laptop|192.168.65.1|192.168.255.10
login_success|1784447444|joe-laptop|192.168.65.1|192.168.255.6
logout|1784447707|joe-laptop|192.168.65.1|192.168.255.6|263
login_success|1784622927|joe-laptop2|192.168.65.1|192.168.255.6
login_success|1784623732|joe-laptop2|192.168.65.1|192.168.255.6
logout|1784623787|joe-laptop2|192.168.65.1|192.168.255.6|55
login_success|1784624293|joe-laptop2|192.168.65.1|192.168.255.6
login_success|1784626651|joe-laptop2|192.168.65.1|192.168.255.6
login_success|1784627649|joe-laptop2|192.168.65.1|192.168.255.6
logout|1784627942|joe-laptop2|192.168.65.1|192.168.255.6|293
login_success|1785620580|joe-laptop2|192.168.65.1|192.168.255.6
login_success|1785624459|user1|192.168.65.1|192.168.255.6
logout|1785624469|user1|192.168.65.1|192.168.255.6|10
login_success|1785625059|user1|192.168.65.1|192.168.255.6
logout|1785625092|user1|192.168.65.1|192.168.255.6|33
login_success|1785625186|user1|192.168.65.1|192.168.255.6
logout|1785625203|user1|192.168.65.1|192.168.255.6|18
login_success|1785625203|user1|192.168.65.1|192.168.255.6
joeefelix@Joes-MacBook-Air access_logs %

```

Figure 5. Full contents of events.log, showing successful login and logout events recorded by the connect and disconnect hooks across four device identities.

Figure 5 is the raw input to the normalisation layer: thirty-one login and logout lines recorded automatically by client-connect.sh and client-disconnect.sh, with no manual editing. The sequence visibly follows joe's original sessions, the two device re-issues, and the beginning of user1's test sessions, giving the detection engine a genuinely mixed history to evaluate rather than a single uniform block of test traffic.

```

~/Documents/access_logs -- -zsh
joeefelix@Joes-MacBook-Air access_logs % python3 normalize_openvpn_logs.py ~/openvpn-data/events.log vpn_logs.csv
Using local GeoLite2 database: GeoLite2-City.mmdb
Wrote 32 normalized events to vpn_logs.csv
Now run: python3 analyze_logs.py vpn_logs.csv
joeefelix@Joes-MacBook-Air access_logs % python3 analyze_logs.py vpn_logs.csv

Loaded 32 log entries from vpn_logs.csv

Total anomalies flagged: 8
  unusual_country: 1
  off_hours_access: 7

Wrote anomalies.json and dashboard_data.js
joeefelix@Joes-MacBook-Air access_logs %
joeefelix@Joes-MacBook-Air access_logs % open dashboard.html

joeefelix@Joes-MacBook-Air access_logs %

```

Figure 6. Normalisation of raw events into the CSV schema, followed by the detection engine flagging eight anomalies across two detector types.

Figure 6 shows `normalize_openvpn_logs.py` using the local GeoLite2 database to enrich all thirty-two events, followed by `analyze_logs.py` loading that CSV and reporting eight anomalies: one unusual_country finding and seven off_hours_access findings. No brute-force, impossible-travel, or long-session findings were produced in this run; Section 6 explains why each of those three stayed silent rather than treating their absence as a shortfall.

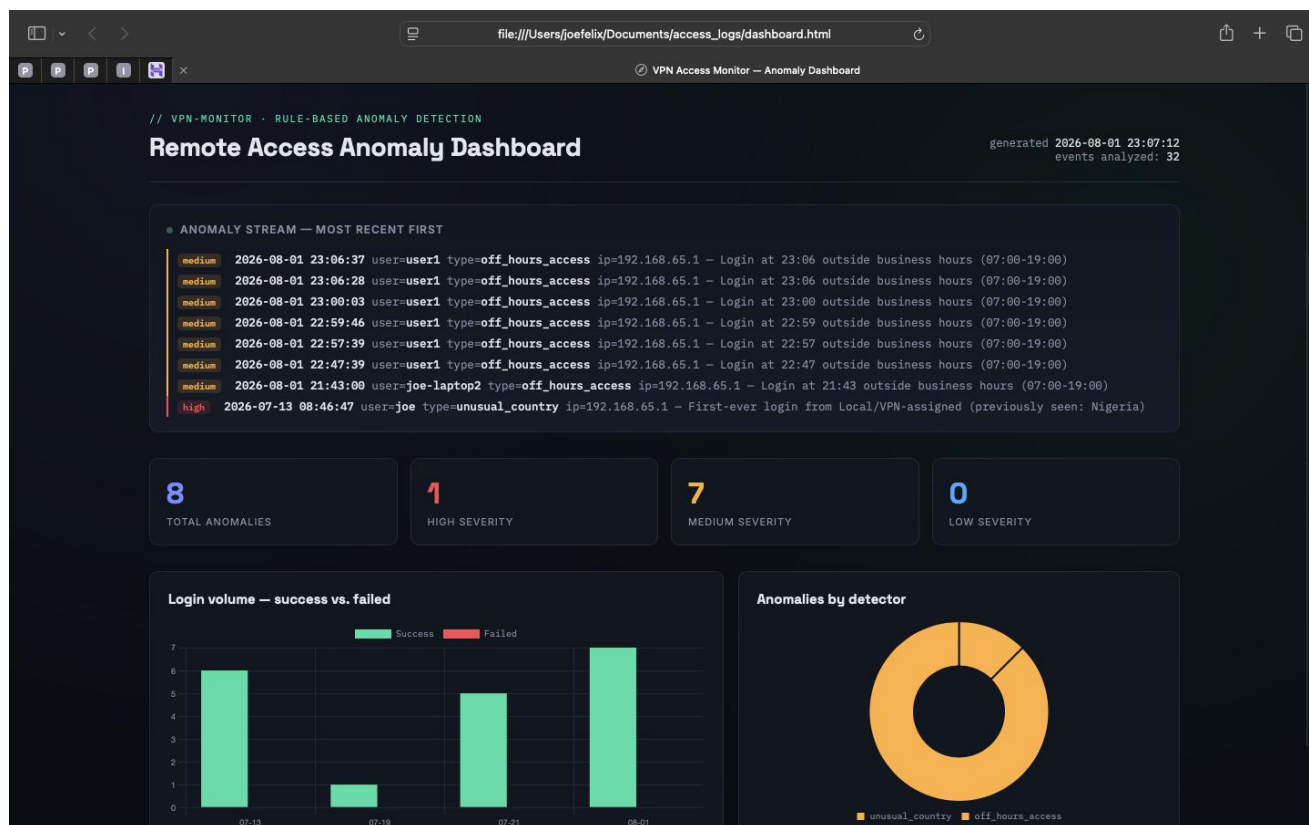


Figure 7. Dashboard anomaly stream and summary counters, showing eight total anomalies split across one high-severity and seven medium-severity findings.

Figure 7 shows the dashboard's top section: the anomaly stream lists every finding most-recent-first with its severity, type, user, source IP, and a short detail string, and the counters below confirm the same one high, seven medium, zero low split reported at the command line in Figure 6. The login-volume chart beneath it breaks total events down by day across the four days that produced traffic.

SEVERITY	TYPE	USER	SOURCE IP	TIMESTAMP	DETAIL
MEDIUM	off_hours_access	user1	192.168.65.1	2026-08-01 23:06:37	Login at 23:06 outside business hours (07:00-19:00)
MEDIUM	off_hours_access	user1	192.168.65.1	2026-08-01 23:06:28	Login at 23:06 outside business hours (07:00-19:00)
MEDIUM	off_hours_access	user1	192.168.65.1	2026-08-01 23:00:03	Login at 23:00 outside business hours (07:00-19:00)
MEDIUM	off_hours_access	user1	192.168.65.1	2026-08-01 22:59:46	Login at 22:59 outside business hours (07:00-19:00)
MEDIUM	off_hours_access	user1	192.168.65.1	2026-08-01 22:57:39	Login at 22:57 outside business hours (07:00-19:00)
MEDIUM	off_hours_access	user1	192.168.65.1	2026-08-01 22:47:39	Login at 22:47 outside business hours (07:00-19:00)
MEDIUM	off_hours_access	joe-laptop2	192.168.65.1	2026-08-01 21:43:00	Login at 21:43 outside business hours (07:00-19:00)
HIGH	unusual_country	joe	192.168.65.1	2026-07-13 08:46:47	First-ever login from Local/VPN-assigned (previously seen: Nigeria)

Figure 8. Dashboard flagged-connections table with severity, type, user, source IP, and timestamp for every anomaly, alongside the detector-type filter control.

Figure 8 shows the dashboard's flagged-connections table and the donut chart summarising anomalies by detector. The filter buttons above the table let an analyst narrow the view to a single detector type, which becomes more useful as the log grows and the anomaly stream gets longer than fits on one screen.

The single `unusual_country` finding is worth reading carefully rather than at face value. It fired because joe's very first recorded session came from a real external address that geolocated to Nigeria, and the next session came from the local, Docker-assigned gateway address, which the detector correctly read as a different country from the user's established baseline. In a production deployment this would be a legitimate flag; in this lab, it is also a direct symptom of the network-address-translation limitation noted in Section 3.3, since every later session from every identity shows the same local address regardless of where the physical device actually was. That distinction, a detector firing on real data for a reason that is partly an artefact of the lab network, is discussed further in Section 6.

The seven `off_hours_access` findings are more straightforward: all seven correspond to genuine sessions, mostly `user1` connecting repeatedly between 21:00 and 23:00 while this pipeline was itself being built and debugged. These were not injected for demonstration purposes; they are an accurate, if mundane, example of the detector doing exactly what it is designed to do.

6. Analysis and Recommendations

6.1 Reading the results together

The result worth emphasising is not simply that eight anomalies were flagged, but that the two detectors which fired did so on genuine log entries, while the three that stayed silent did so for reasons that check out against the underlying data rather than because the engine failed to run correctly. The brute-force detector found no username-and-IP pair with five or more failed logins inside a 15-minute window, which is accurate: `watch_failed_auth.sh` never recorded a failure of that kind during this test period. The impossible-travel detector needs two successful logins with populated coordinates far enough apart to imply an unreasonable travel speed, and because Docker's address translation means only joe's very first session carried a real, non-local IP, there was never a second distant point available to compare it against. The long-session detector needs a session both above eight hours and at least three times that user's median, and the longest session in the dataset, twelve point three minutes on joe's first Nigerian login, was nowhere close to that floor. A detector that stays quiet because its condition genuinely was not met is behaving correctly, and that is as much a result worth reporting as a detector that fires.

6.2 The central limitation

The most consequential finding of this project is not one of the five anomaly types but the network-level limitation that shaped how several of them behaved: running the OpenVPN server inside Docker Desktop on a laptop means every client connection is network-address-translated to the same local gateway address before it ever reaches the server. Two of the five detectors, impossible travel and, to a lesser extent, unusual country, depend on the server seeing each client's real internet-facing address. Behind Docker Desktop's NAT, that information is simply not available, regardless of how well the detection logic itself is written.

This is a lab-environment artefact rather than a flaw in the detection engine or the normalisation schema, and it would not occur in a deployment where the OpenVPN server sits on a routable address, whether a cloud VM's public IP, a colocated machine, or a home server behind a port-forwarded router rather than a container platform's internal NAT. That distinction is worth stating plainly rather than glossing over, since it is exactly the kind of gap between a lab result and a production result that an interview question is likely to probe.

6.3 Recommendations

- Deploy the OpenVPN server on a routable host, not behind a local container platform's NAT, before relying on the impossible-travel or unusual-country detectors in anything beyond a lab setting.
- Replace or supplement the regex-based `watch_failed_auth.sh` with a more deterministic failure signal once the target authentication backend is fixed, since pattern-matching log wording is, by its own documentation, a heuristic that varies across authentication methods.

- Back the unusual-country and long-session baselines with a small persistent store, such as SQLite, so each user's history survives across runs instead of being rebuilt from whatever happens to be in the current log file.
- Wire alerting, such as an email or Slack notification, off high-severity findings specifically, rather than requiring an analyst to open the dashboard to notice anything has fired.
- Build a short, deliberate test harness that induces a handful of real failed logins during development, so the brute-force detector is exercised and confirmed at least once before being relied on against live traffic.

6.4 Why the rule-based approach held up

Every one of the eight findings in Section 5, and every one of the three detectors that produced no findings, can be traced back to a single readable condition in `analyze_logs.py`. That traceability is the practical payoff of choosing explainable rules over a statistical model for a first monitoring pass: when a finding needs to be explained, or defended, or tuned because it produced a false positive, the answer is always a specific threshold in a specific function, not a weight inside a model nobody on the team can inspect.

7. Conclusion

This project set out to build a working, explainable monitoring layer over an OpenVPN deployment, and the resulting pipeline does what it was designed to do: it turns a raw connection log into a set of geolocated, timestamped events, applies five threshold-based detectors grounded in recognised attacker techniques, and surfaces the results on a dashboard an analyst can actually read. Tested against real, if modest, logged history rather than a purely synthetic dataset, two detectors fired correctly and three stayed silent for reasons the underlying data supports, which together demonstrate that the engine behaves the way its rules say it should rather than merely producing alerts.

The most important limitation surfaced along the way was not a defect in the detection logic but a property of the lab environment itself: running the server behind Docker Desktop's network address translation hides the real source address of every client, which specifically defeats the two detectors that depend on knowing where a connection genuinely originated. That limitation, and the straightforward fix of deploying on a routable host instead, is worth carrying forward into any future extension of this work.

More broadly, this project is a small demonstration of a point that applies well beyond a class lab: a VPN gateway that only keeps a connection log, and nothing that reads it, leaves a real and common gap open. A rule-based layer sitting quietly over that same log, cheap to build, cheap to explain, and cheap to tune, closes a meaningful part of that gap without requiring the organisation to adopt a full SIEM platform or a machine-learning pipeline it may not have the data or the staff to maintain.

References

MITRE ATT&CK, "External Remote Services, Technique T1133," MITRE Corporation. [Online]. Available: <https://attack.mitre.org/techniques/T1133/>

MITRE ATT&CK, "Valid Accounts, Technique T1078," MITRE Corporation. [Online]. Available: <https://attack.mitre.org/techniques/T1078/>

MITRE ATT&CK, "Brute Force, Technique T1110," MITRE Corporation. [Online]. Available: <https://attack.mitre.org/techniques/T1110/>

Center for Internet Security, "CIS Critical Security Controls, Version 8," CIS, 2021.

M. Souppaya and K. Scarfone, "Guide to Enterprise Telework, Remote Access, and Bring Your Own Device (BYOD) Security," NIST Special Publication 800-46 Revision 2, National Institute of Standards and Technology, 2016.

OpenVPN Inc., "OpenVPN Community Resources." [Online]. Available: <https://openvpn.net/community-resources/>

MaxMind Inc., "GeoLite2 Free Geolocation Data." [Online]. Available: <https://www.maxmind.com/en/geolite2/geoip2-databases>

Appendix A: Selected Script Listings

Full source for every script referenced in this report, including `normalize_openvpn_logs.py` and `analyze_logs.py` in their entirety, is included in the project's GitHub repository listed on the cover page. The listings below cover the two hook scripts and the failed-login watcher in full, since they are short enough to review directly.

A.1 client-connect.sh

```

15 LOGFILE="/etc/opencvn/events.log"
16 mkdir -p "$(dirname "$LOGFILE")"
17
18 # Fields: event|epoch_seconds|common_name(username)|trusted_ip|assigned_vpn_ip
19 echo "login_success|$(date +%s)|${common_name}|${trusted_ip}|${ifconfig_pool_remote_ip}" >> "$LOGFILE"
20
21 exit 0
22

```

A.2 client-disconnect.sh

```

9 LOGFILE="/etc/opencvn/events.log"
10 mkdir -p "$(dirname "$LOGFILE")"
11
12 # Fields: event|epoch_seconds|common_name|trusted_ip|assigned_vpn_ip|duration_seconds
13 echo "logout|$(date +%s)|${common_name}|${trusted_ip}|${ifconfig_pool_remote_ip}|${time_duration}" >> "$LOGFILE"
14
15 exit 0
16

```

A.3 watch_failed_auth.sh

```

17 SRC_LOG="/var/log/opencvn/opencvn.log"
18 OUT_LOG="/var/log/opencvn/events.log"
19 mkdir -p "$(dirname "$OUT_LOG")"
20
21 tail -Fn0 "$SRC_LOG" | while read -r line; do
22     if echo "$line" | grep -qE "auth.?fail|verify error|tls.?error|authentication failed"; then
23         ip=$(echo "$line" | grep -oE '[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}' | head -n1)
24         user=$(echo "$line" | grep -oP "(?<=DN=)(^,\\s)+|(?!client ')[^']*" | head -n1)
25         [ -z "$user" ] && user="unknown"
26         [ -z "$ip" ] && ip="unknown"
27         echo "login_failed|$(date +%s)|${user}|${ip}" >> "$OUT_LOG"
28     fi
29 done
30

```

Appendix B: Raw Event Log

The complete, unedited contents of events.log at the time the detection engine was run, corresponding to the input shown in Figure 5:

```

~/.Documents/access_logs -- -zsh
joefelix@Joes-MacBook-Air access_logs % cat ~/openvpn-data/events.log

logout|1783931025|joe|192.168.65.1|192.168.255.6|239
login_success|1783931026|joe|184.27.140.133|192.168.255.6
logout|1783931763|joe|184.27.140.133|192.168.255.6|737
login_success|1783932407|joe|192.168.65.1|192.168.255.6
logout|1783932538|joe|192.168.65.1|192.168.255.6|131
login_success|1783932538|joe|192.168.65.1|192.168.255.6
logout|1783932598|joe|192.168.65.1|192.168.255.6|60
login_success|1783932598|joe|192.168.65.1|192.168.255.6
login_success|1783934089|joe|192.168.65.1|192.168.255.6
logout|1783934511|joe|192.168.65.1|192.168.255.6|422
login_success|1783935140|joe-laptop|192.168.65.1|192.168.255.10
login_success|1784447444|joe-laptop|192.168.65.1|192.168.255.6
logout|1784447707|joe-laptop|192.168.65.1|192.168.255.6|263
login_success|1784622927|joe-laptop2|192.168.65.1|192.168.255.6
login_success|1784623732|joe-laptop2|192.168.65.1|192.168.255.6
logout|1784623787|joe-laptop2|192.168.65.1|192.168.255.6|55
login_success|1784624293|joe-laptop2|192.168.65.1|192.168.255.6
login_success|1784626651|joe-laptop2|192.168.65.1|192.168.255.6
login_success|1784627649|joe-laptop2|192.168.65.1|192.168.255.6
logout|1784627942|joe-laptop2|192.168.65.1|192.168.255.6|293
login_success|1785620580|joe-laptop2|192.168.65.1|192.168.255.6
login_success|1785624459|user1|192.168.65.1|192.168.255.6
logout|1785624469|user1|192.168.65.1|192.168.255.6|10
login_success|1785625059|user1|192.168.65.1|192.168.255.6
logout|1785625092|user1|192.168.65.1|192.168.255.6|33
login_success|1785625186|user1|192.168.65.1|192.168.255.6
logout|1785625203|user1|192.168.65.1|192.168.255.6|18
login_success|1785625203|user1|192.168.65.1|192.168.255.6
joefelix@Joes-MacBook-Air access_logs %

```